Comparing DPO and PPO-RLHF for Reinforcement Learning from Preferences

João Pinto Pedro Pinto Tim Arni

REWARD FOCUSED

EPFL

Introduction

Reinforcement Learning from Human Feedback (RLHF) [2] is the standard recipe for aligning large language models (LLMs) [4]. We revisit the two dominant preference-based fine-tuning paradigms and study them in classical RL control settings:

- **PPO-RLHF [6]** – fit a reward model $\hat{r}_\phi$ from preferences, then optimise it with PPO under a KL anchor to a reference policy.
- **Direct Preference Optimisation (DPO) [5]** – skip the reward model and optimise the policy directly from preference pairs in closed form.

Research question. How do the two methods compare in *sample efficiency*, *stability*, and *final performance* as we vary $K \in \{50, 200, 1000\}$ preference pairs across diverse environments?

Contribution. A unified pipeline that (i) generates Bradley–Terry preference datasets from a mid- and an expert-policy, (ii) trains PPO-RLHF / SAC-RLHF and DPO from the *same* mid-policy anchor, and (iii) reports scaling laws on the dataset size K. **Code.** `github.com/pedropintoo/RL-project`.

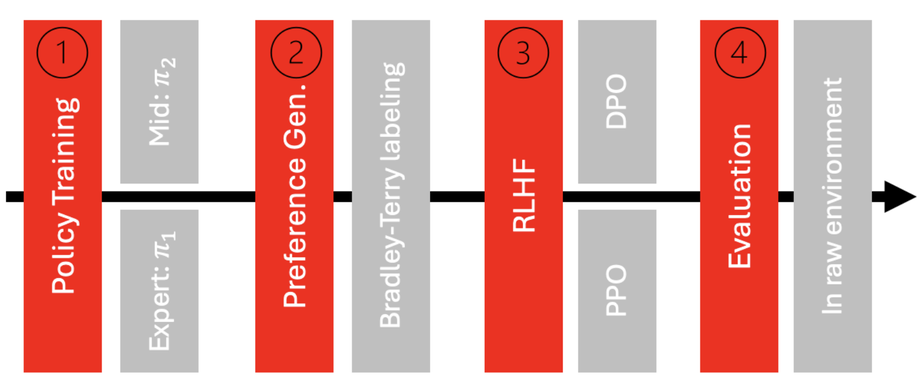


Figure 1. Four-stage pipeline: (1) train a single policy and capture expert (π_1) and mid (π_2) checkpoints; (2) roll out trajectory pairs and label them with Bradley–Terry probabilities; (3) fine-tune from π_2 with either DPO or PPO/SAC-RLHF; (4) evaluate the resulting policy on the raw environment reward.

Environments

Three Gymnasium [3] environments span action space and reward density (see Fig. 2 and Table 1). For MountainCarContinuous, standard PPO collapses to the do-nothing local optimum, so we use SAC with gSDE.

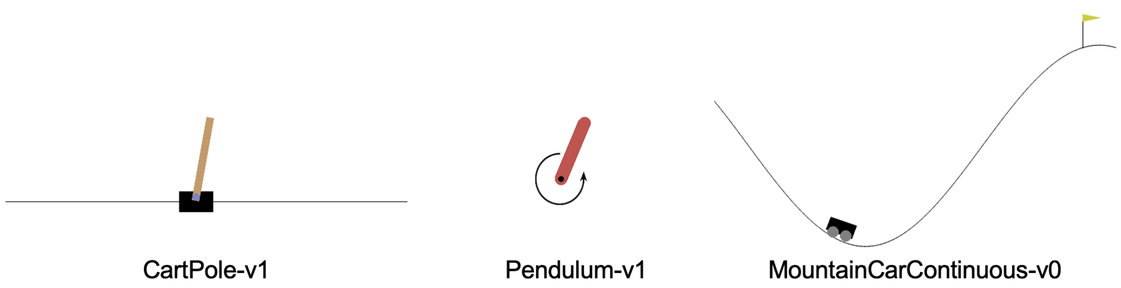


Figure 2. The three Gymnasium environments used in this study.

Environment	Action	Reward	Algorithm	Expert return
CartPole-v1	discrete	dense	PPO	500
Pendulum-v1	continuous	dense	PPO	−261
MountainCarContinuous-v0	continuous	sparse	SAC (gSDE)	94.3

Table 1. Per-environment setup and expert returns.

Preference Data Generation

For each environment we run a single PPO (or SAC) training and capture two checkpoints on the *same* trajectory:

- π_1 – expert policy (final checkpoint).
- π_2 – mid-anchor, saved at midpoint return between random and expert.

For each $K \in \{50, 200, 1000\}$ and 5 seeds we roll out stochastic trajectory pairs $(\tau_1 \sim \pi_1, \tau_2 \sim \pi_2)$ and label them with the Bradley–Terry probability [1]

$$p(\tau_1 \succ \tau_2) = \sigma\big(R(\tau_1) - R(\tau_2)\big),$$

using the numerically stable sigmoid form. Binary labels are sampled as $y \sim \text{Bernoulli}(p)$.

Algorithms: PPO-RLHF

Fit a reward model (RM) $\hat{r}_\phi(s,a)$ via Bradley–Terry negative log-likelihood: $\mathcal{L}_{\text{RM}} = -\log\sigma\big(\hat{R}_\phi(\tau^+) - \hat{R}_\phi(\tau^-)\big)$, then run PPO from π_2 maximising

$$\mathbb{E}_{s,a\sim\pi_\theta}[\hat{r}_\phi(s,a)] \;-\; \beta\,\mathbb{E}_s\big[\text{KL}(\pi_\theta \| \pi_{\text{ref}})\big], \text{ with } \pi_{\text{ref}} = \pi_2.$$

SAC-RLHF (sparse reward): For MountainCarContinuous-v0 we replace PPO with SAC + gSDE in both data generation and the RLHF fine-tuning stage to keep our mid-policy alignment pipeline consistent.

Algorithms: DPO

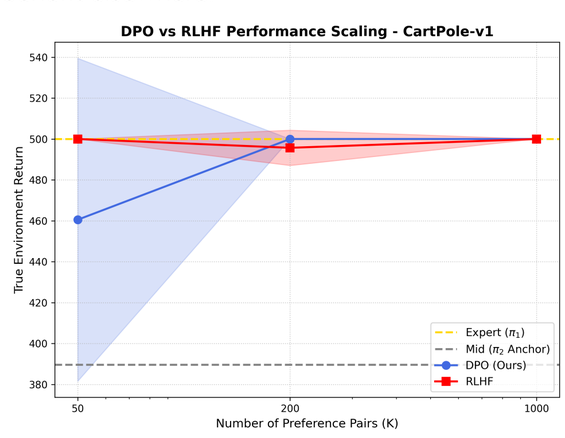
Skip the reward model and optimise the closed-form policy loss derived from the same Bradley–Terry model:

$$\mathcal{L}_{\text{DPO}} = -\log\sigma\Big(\beta\big[\log\tfrac{\pi_\theta(\tau^+)}{\pi_{\text{ref}}(\tau^+)} - \log\tfrac{\pi_\theta(\tau^-)}{\pi_{\text{ref}}(\tau^-)}\big]\Big) + \lambda_{\text{KL}}\,\text{KL}(\pi_\theta\|\pi_{\text{ref}}).$$

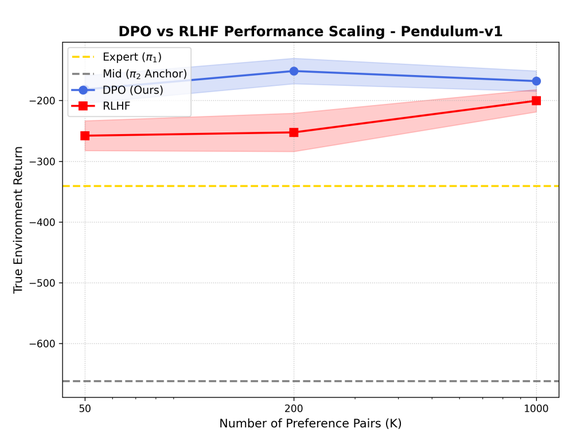
The explicit KL term mitigates overfitting on finite offline data. Here $\pi_{\text{ref}} = \pi_2$ (mid-policy).

Results

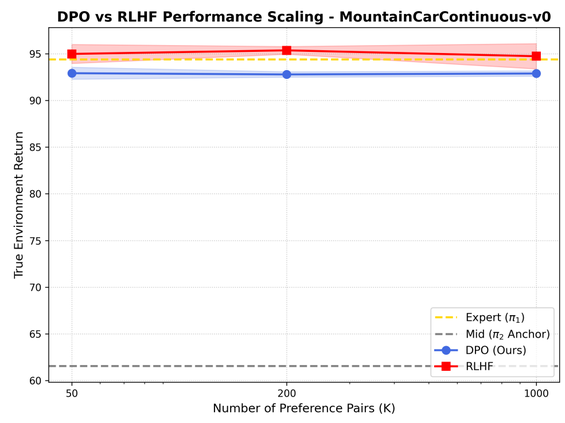
Hyperparameter sensitivity. DPO is more sensitive than PPO-RLHF. For **DPO**, discrete actions allow $\lambda_{\text{KL}} = 0.1$ (bounded KL), continuous Gaussian policies need $\lambda_{\text{KL}} \in [0.2, 0.4]$ and LR 10^{-3}, and sparse rewards require stronger anchoring ($\lambda_{\text{KL}} = 0.4$, $\beta = 0.4$). For **PPO-RLHF**, ablating $\beta \in \{0.01, 0.1, 0.5, 2.0\}$ selects $\beta = 0.1$ as best; $\beta = 2.0$ suppresses learning across environments, $\beta = 0.01$ causes high variance on Pendulum at K=50 (std= 179), and $\beta = 0.5$ underperforms on Pendulum while offering no benefit elsewhere.



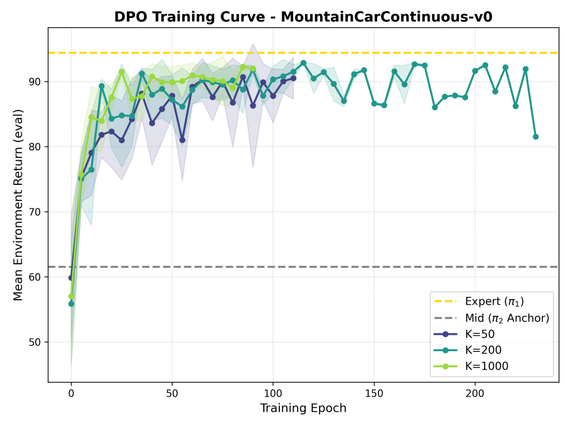
CartPole-v1 (DPO vs. PPO-RLHF)



Pendulum-v1 (DPO vs. PPO-RLHF)



MountainCarCont-v0 (DPO vs. SAC-RLHF)



DPO Training Curve

Figure 3. **Top and Bottom-Left:** True environment return vs. number of preference pairs K (in log scale). **Bottom-Right:** DPO training curves on MountainCarContinuous-v0, showing mean environment return over training epochs for varying K. Across all plots, dashed lines indicate expert (π_1) and mid-anchor (π_2) performance, and bands span $\pm 1\,\sigma$ over 5 seeds.

		K=50	K=200	K=1000
CartPole	PPO-RLHF	$\mathbf{500.0 \pm 0.0}$	495.7 ± 8.6	$\mathbf{500.0 \pm 0.0}$
	DPO	460.5 ± 79.0	$\mathbf{500.0 \pm 0.0}$	$\mathbf{500.0 \pm 0.0}$
Pendulum	PPO-RLHF	-257.8 ± 24.6	-252.4 ± 31.6	-200.3 ± 18.4
	DPO	$\mathbf{-181.7 \pm 22.7}$	$\mathbf{-151.4 \pm 21.0}$	$\mathbf{-168.0 \pm 16.8}$
MountainCar	SAC-RLHF	$\mathbf{95.0 \pm 1.0}$	$\mathbf{95.4 \pm 0.4}$	$\mathbf{94.7 \pm 1.4}$
	DPO	92.9 ± 0.6	92.8 ± 0.3	92.9 ± 0.3

Table 2. **True environment return (mean ± std, 5 seeds).** Best result in each cell is bolded. On CartPole, PPO-RLHF wins at K=50, DPO wins at K=200, and both tie at K=1000. DPO outperforms PPO-RLHF on Pendulum at every K. SAC-RLHF dominates DPO on MountainCar at all K, marginally exceeding the expert return (94.3) at every dataset size. Notably, both methods surpass the Pendulum expert.

		Phase 1 (RM)		Phase 2 (Policy)		PPO-RLHF Total		DPO	
	K	T (s)	Steps	T (s)	Steps	T (s)	Steps	T (s)	Steps
CartPole	50	0.7±0.5	250	90.8±4.3	8'128±384	91.6±4.3	8'378±384	**39.2±11.0**	**3'120±2'488**
	200	1.7±0.1	1'000	98.0±11.3	8'576±1'364	99.7±11.4	9'576±1'364	**62.6±2.8**	**1'750±105**
	1000	9.0±0.5	5'000	87.3±6.1	7'936±314	**96.3±5.9**	12'936±314	222.7±6.9	**2'112±0.0**
Pendulum	50	1.2±0.2	500	151.6±14.9	25'856±1'803	152.8±14.9	26'356±1'803	**31.8±6.4**	**3'200±616**
	200	3.9±0.7	2'000	177.2±33.7	30'656±5'729	181.1±33.4	32'656±5'729	**76.2±20.9**	**3'100±758**
	1000	16.3±0.6	10'000	169.2±39.1	30'016±6'897	**185.5±39.7**	40'016±6'897	304.6±77.9	**3'744±887**
MountainCar	50	2.6±0.2	1'250	220.6±36.8	29'990±6'194	223.2±36.7	31'240±6'194	**105.2±31.0**	**4'350±1405**
	200	11.8±3.8	5'000	212.4±26.5	27'194±4'403	224.2±28.5	32'194±4'403	**168.3±69.0**	**3'700±1377**
	1000	46.6±1.3	25'000	227.5±23.4	26'989±3'994	**274.1±22.9**	51'989±3'994	587.0±231.3	**4'832±1953**

Table 3. **Computational cost.** Best result per row is bolded; Phases 1 and 2 are PPO-RLHF stages (reward model and policy training). PPO-RLHF needs 3–11× more gradient steps than DPO. DPO's sample efficiency translates to a 1.3–5× wall-clock advantage at small K; at K=1000, however, DPO's per-epoch cost scales with dataset size which erases this advantage and makes PPO-RLHF 1.6–2.3× faster across all three environments. PPO-RLHF Phase 2, which is the most time-consuming phase, is not affected this way because its per-step cost is independent of K.

Discussion & Conclusion

Take-aways

- **CartPole.** PPO-RLHF wins at K=50 (500±0 vs. 461±79); DPO is unreliable at low data due to high variance. Both methods reach the expert return at $K \geq 200$.
- **Pendulum.** DPO outperforms PPO-RLHF at every K: at K=50 DPO already beats PPO-RLHF's best at K=1000, implying ∼20× greater sample-efficiency. Both methods surpass the expert.
- **MountainCar (sparse).** SAC-RLHF dominates (≈95 vs. ≈92.9). DPO still reaches near-expert performance with low variance ($\sigma \leq 0.6$), but the explicit reward model better exploits the sparse signal.
- **Headline.** DPO wins on dense continuous rewards and is competitive on discrete environments at sufficient data. An explicit reward model retains an edge on sparse-reward tasks.

When DPO wins – and when it does not

- **Dense rewards (Pendulum, CartPole).** PPO-RLHF's reward-model approximation compounds errors through PPO updates; DPO optimises preferences directly.
- **Sparse rewards (MountainCar).** The reward model acts as a *signal densifier*: $\hat{r}_\phi$ provides a smooth target even when episode rewards are near-zero. DPO, operating on trajectory log-ratios, loses this benefit and trails SAC-RLHF slightly.

Caveat: tuning cost

DPO's competitiveness comes at a tuning price. A misaligned λ_{KL} collapses training – the continuous and sparse settings each needed their own $(\lambda_{\text{KL}}, \beta, \eta)$. For PPO-RLHF, once $\beta = 0.1$ is fixed, it transfers across environments with minimal re-tuning. We observe a *trade-off* between sample efficiency and tuning effort.

References

- Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.
- Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *NeurIPS*, 2017.
- Farama Foundation. Gymnasium: A standard api for reinforcement learning environments. `https://gymnasium.farama.org`, 2023.
- Long Ouyang, Jeff Wu, Xu Jiang, et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *NeurIPS*, 2023.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.